

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Державний вищий навчальний заклад
“Ужгородський національний університет”
Факультет інформаційних технологій
Кафедра інформаційні управляючі системи та технології

Реєстраційний № _____

Дата _____

Пацкан Василь Михайлович
студент 5 курсу
заочної форми навчання
залікова книжка № ____

ЗВІТ

ПРО ПРОХОДЖЕННЯ ПЕРЕДДИПЛОМНОЇ ПРАКТИКИ

Спеціальність "Комп'ютерні науки"

Рекомендовано до захисту з оцінкою:

" _____ " _____

Підпис керівника практики

" ____ " _____ 2019 р.

Керівник
практики від ВУЗу
ст. викл. Копча-
Горячкіна Г.Е.

Робота захищена " ____ " _____ 2019 р. з оцінкою " _____ "

Підписи членів комісії _____

Ужгород – 2019

Зміст

Вступ.....	3
1. Відомості про базу практики.....	4
2. Загальні теоретичні відомості.....	5
Висновки.....	20
Список використаної літератури.....	21

Вступ

Метою переддипломної практики є набуття професійних навичок щодо комп'ютерного моделювання, дослідження різних методів розв'язання задачі про призначення та їх порівняльний аналіз.

Основними завданнями практики полягає у дослідженні різних методів розв'язання задачі про призначення та їх порівняльний аналіз. В роботі розглянуто такі методи як угорський алгоритм та перебір.

Виробнича практика проходила на базі ПП Руцак П.Ю..

Етапи проходження практики:

- ознайомлення з базою практики;
- ознайомлення з технікою безпеки;
- опрацювання літератури згідно теми кваліфікаційної роботи;
- розробка і опис алгоритмів розв'язання задачі про призначення;
- оформлення звіту та підготовка презентації про проходження практики;
- завершення проходження практики.

Інформація про базу проходження практики

Основний вид діяльності – розробка програмного забезпечення. Підприємство надає послуги по міжнародному і внутрішньому ринках для автоматизації бізнес-процесів підприємств. Основними напрямками в області розробки програмного забезпечення є:

- розробка ПО для автоматизації бізнес процесів;
- розробка і впровадження баз даних для збору і обробки інформації.

Компанія дбає про своїх працівників та приділяє багато уваги їхньому професійному та кар'єрному розвитку. Для цього було створена та розроблена «кар'єрна мапа», яка містить чіткі вимоги щодо подальшого руху кар'єрними сходами.

Підприємство ПП Рушак П.Ю. за формою власності є приватним підприємством. Підприємство діє згідно з положеннями Статуту, а також діючого законодавства України. Підприємство є юридичною особою; воно володіє, користується та розпоряджається майном, що йому належить, має самостійний баланс, поточні та інші рахунки.

Розробка програмного забезпечення буде корисна тим організаціям і приватним особам, які не можуть використовувати готове програмне забезпечення унаслідок його незручності, складності, надмірної або недостатньої функціональності, а також в тих випадках, коли відповідного програмного забезпечення просто не існує. У сучасних організаціях все частіше виникає необхідність розробки розподілених систем автоматизації для вирішення завдань об'єднання даних і отримання звітних звітів про роботу компанії. Розробка програм на замовлення дозволяє створювати спеціалізовані системи збору і управління інформацією, максимально використовуючи існуючу інфраструктуру і повністю враховуючи особливості організації бізнес-процесів в конкретній компанії.

ТЕОРЕТИЧНА ЧАСТИНА

Припустимо, що є різні роботи і механізми, кожний з яких може виконувати будь-яку роботу, але з неоднаковою ефективністю. Продуктивність кожного i -го механізму при виконанні j -тої роботи позначимо c_{ij} , $i = 1, \dots, n$; $j = 1, \dots, n$. Потрібно так розподілити механізми по роботах, щоб сумарний ефект від їхнього використання був максимальний. Таке завдання називається завданням вибору або завданням про призначення.

Формально вона записується так. Необхідно вибрати таку послідовність елементів з матриці

$$C = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1n} \\ c_{21} & c_{22} & \dots & c_{2n} \\ \dots & \dots & \dots & \dots \\ c_{n1} & c_{n2} & \dots & c_{nn} \end{bmatrix}$$

щоб сума $\sum_{k=1}^n c_{kj_k}$ була максимальна й при цьому з кожного рядка й стовпця був обраний тільки один елемент.

Математична модель:

Нехай є множина позицій $P = \{p_j\}_{j=1}^n$ і множина елементів $E = \{e_i\}_{i=1}^n$, кожний з яких може займати тільки одну позицію. У кожній позиції може розміститися тільки один елемент. Кожний варіант розміщення i -го елемента в j -й позиції має вартість c_{ij} . Необхідно розмістити всі елементи таким чином, щоб сумарна вартість була мінімальною. Інакше кажучи, треба знайти таку перестановку елементів, що задається відображенням $\Phi: E \rightarrow P$, при якій сума мінімальна

$$\sum_{i=1}^n c_{\Phi(i), i} \rightarrow \min \quad (1.1)$$

Число можливих варіантів перестановок дорівнює $n!$.

У різних задачах елементи і позиції можуть трактуватися як працівники й роботи (власне задача про призначення); верстати й оброблювані на них деталі; роботи й строки або черговість їхнього виконання; клієнти й сервіси (у магазині

або в розподіленому обчислювальному середовищі); електрорадіоелементи й монтажні модулі (задача компоновання елементів конструкції), електрорадіоелементи й монтажні площадки (задача розміщення елементів на друкованій платі).

Якщо в конкретній задачі число елементів не дорівнює числу позицій, то можна ввести фіктивні елементи або фіктивні позиції з досить великою вартістю (щоб при рішенні мінімізаційної задачі фіктивні елементи або позиції гарантовано не входили в оптимальне рішення).

Задачу про призначення можна розглядати з позицій лінійного програмування.

Уведемо матрицю призначень X розміром $n \times n$, елементи якої розраховуються за правилом

$$x_{ij} = \begin{cases} 1, & \text{якщо елемент } i \text{ назначений в позицію } j \\ 0, & \text{якщо в іншому випадку} \end{cases}.$$

Одержуємо наступну задачу лінійного програмування: мінімізувати

$$z = \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij}$$

при виконанні умов:

$$\sum_{j=1}^n x_{ij} = 1; \quad i = 1, 2, \dots, n$$

(кожний елемент призначений в одну й тільки в одну позицію);

$$\sum_{i=1}^n x_{ij} = 1; \quad j = 1, 2, \dots, n$$

(для кожної позиції є один і тільки один призначений у неї елемент);

$$x_{ij} \in \{0; 1\}$$

(невідомі x_{ij} можуть приймати тільки значення 0 і 1).

Таким чином, задачу про призначення можна вирішувати методами цілочисельного (дискретного) лінійного програмування.

Також задача про призначення є частинним випадком транспортної задачі, у якій призначувані працівники є пунктами відправлення, а роботи - пунктами призначення, тільки в цьому випадку величини, що характеризують

транспортну задачу, попиту та пропозиції рівні 1. Тому задачу про призначення можна вирішувати тими ж методами, що й транспортну задачу. Але той факт, що величини попиту та пропозиції рівні 1, дозволили розробити більше ефективний спосіб рішення - так званий Угорський метод.

Під назвою «транспортна задача» об'єднується широкий круг завдань з єдиною математичною моделлю. Дані задачі відносяться до задач лінійного програмування і можуть бути вирішені симплекс-методом. Проте матриця системи обмежень транспортного завдання настільки своєрідна, що для її вирішення розроблені спеціальні методи. Ці методи, як і симплексний метод, дозволяють знайти початковий опорний розв'язок, а потім, покращуючи його, отримати оптимальний розв'язок [5].

У загальній постановці транспортна задача полягає у пошуку оптимального плану перевезень деякого однорідного вантажу з m баз $A_1, A_2, \dots, A_m$ n споживачам $B_1, B_2, \dots, B_n$.

Розрізняють два типи транспортних завдань: по *критерію вартості* (план перевезень оптимальний, якщо досягнутий мінімум витрат на його реалізацію) і по *критерію часу* (план оптимальний, якщо на його реалізацію витрачається мінімум часу).

Позначимо кількість вантажу, що є на кожній з m баз (запаси), відповідно $a_1, a_2, \dots, a_m$, а загальна кількість вантажу, що є в наявності – a :

$$a = a_1 + a_2 + \dots + a_m; \quad (1.2)$$

замовлення кожного із споживачів (потреби) позначимо відповідно $b_1, b_2, \dots, b_n$ а загальна кількість потреб – Φ :

$$b = b_1 + b_2 + \dots + b_n, \quad (1.3)$$

Тоді за умови

$$a = b \quad (1.4)$$

ми маємо *закриту модель*, а за умови

$$a \neq b \quad (1.5)$$

– *відкриту модель транспортної задачі*.

Очевидно, що у разі закритої моделі, весь вантаж, що є в наявності, розвозиться повністю, і всі потреби замовників повністю задоволені; у разі ж відкритої моделі або всі замовники задоволені і при цьому на деяких базах залишаються надлишки вантажу ($a > b$) або весь вантаж виявляється витраченим, хоча потреби повністю не задоволені ($a < b$).

Так само існують одно етапні моделі завдань, де перевезення здійснюється безпосередньо від, наприклад, бази або заводу виробника до споживача, і двох етапні, де між ними є «перевалочний пункт», наприклад – склад.

План перевезень з вказівкою запасів і потреб зручно записувати у вигляді наступної таблиці, яку називають *таблицею перевезень*:

всі рівняння системи (1.6) можуть бути розділені на дві групи: перша група з m перших рівнянь («горизонтальні» рівняння) і другої групи з n решти рівнянь («вертикальні» рівняння). У кожному з горизонтальних рівнянь містяться невідомі з одним і тим же першим індексом (вони утворюють один рядок матриці перевезень), в кожному з вертикальних рівнянь містяться невідомі з одним і тим же другим індексом (вони утворюють один стовпець матриці перевезень). Таким чином, кожна невідома зустрічається в системі (1.6) двічі: у одному і лише одному горизонтальному і в одному і лише одному вертикальному рівняннях.

Така структура системи (1.6) дозволяє легко встановити її ранг. Дійсно, покажемо, що сукупність невідомих, утворюючих перший рядок і перший стовпець матриці перевезень, можна прийняти як базис. При такому виборі базису, принаймні, один з двох їх індексів рівний одиниці, а, отже, вільні невідомі визначаються умовою $i \geq 2, j \geq 2$.

$$\left. \begin{array}{l} x_{11} + x_{12} + \dots + x_{1n} = a_1 \\ x_{21} + \sum_{j \geq 2} x_{2j} = a_2 \\ \dots\dots\dots \\ x_{m1} + \sum_{j \geq 2} x_{mj} = a_m \\ x_{11} + x_{21} + \dots + x_{m1} = b_1 \\ x_{12} + \sum_{i \geq 2} x_{i2} = b_2 \\ \dots\dots\dots \\ x_{1n} + \sum_{i \geq 2} x_{in} = b_n \end{array} \right\} \quad (1.1.7)$$

де символи $\sum_{j \geq 2}$ і $\sum_{i \geq 2}$ означають підсумовування по відповідному індексу. Так,

наприклад

$$\sum_{j \geq 2} x_{2j} = x_{22} + x_{23} + \dots + x_{2n}$$

$$\sum_{i \geq 2} x_{i2} = x_{22} + x_{32} + \dots + x_{m2}$$

При цьому легко відмітити, що під символами такого підсумовування об'єднуються тільки вільні невідомі (тут $i \geq 2 \quad j \geq 2$).

У системі, що розглядається нами, тільки два рівняння, а саме перше горизонтальне і перше вертикальне, містять більше одного невідомого з числа вибраних нами для побудови базису. Виключивши з першого горизонтального рівняння базисні невідомі $x_{12}, x_{13}, \dots, x_{1n}$ за допомогою вертикальних рівнянь, ми отримуємо рівняння

$$x_{11} - \sum_{i \geq 2} x_{i2} - \dots - \sum_{i \geq 2} x_{in} = a_1 - b_2 - b_3 - \dots - b_n$$

або коротше

$$x_{11} - \sum_{i \geq 2, j \geq 2} x_{ij} = a_1 + b_1 - b, \quad (1.9)$$

де символ $\sum_{i \geq 2, j \geq 2} x_{ij}$ означає суму всіх вільних невідомих. Аналогічно,

виключивши з першого вертикального рівняння базисні невідомі $x_{21}, x_{31}, \dots, x_{m1}$ за допомогою горизонтальних рівнянь, ми отримуємо рівняння

$$x_{11} - \sum_{i \geq 2, j \geq 2} x_{ij} = a_1 + b_1 - a \quad (1.10)$$

Оскільки для закритої моделі транспортного завдання $a = b$, то отримані нами рівняння (1.9) і (1.10) однакові і, виключивши з одного з них невідоме x_{11} ми отримаємо рівняння-тотожність $0=0$, яке з системи викреслюється.

Отже, перетворення системи (1.6) звелось до заміни двох рівнянь (першого горизонтального і першого вертикального) рівнянням (1.9). Решта рівнянь залишається незмінними. Система прийняла вигляд

$$(1.11)$$

Для вирішення транспортної задачі необхідно окрім запасів і потреб знати також і тарифи c_{ij} тобто вартість перевезення одиниці вантажу з бази A_i споживачеві B_j .

12

Таблиця 1.2

Пункти Відправле ння	Пункти призначення				Запаси
	B_1	B_2	.	B_n	
A_1	c_{11} x_{11}	c_{12} x_{12}	.	c_{1n} x_{1n}	a_1
A_2	c_{21} x_{21}	c_{22} x_{22}	.	c_{2n} x_{2n}	a_2
.	.	.	.	.	.
A_m	c_{m1} x_{m1}	c_{m2} x_{m2}	.	c_{mn} x_{mn}	a_m
Потреби	b_1	b_2	.	b_n	$a = b$ або $a \neq b$

Сума всіх витрат, тобто вартість реалізації даного плану перевезень, є лінійною функцією змінних x_{ij} :

$$S = c_{11}x_{11} + c_{12}x_{12} + \dots + c_{mn}x_{mn} = \sum c_{ij}x_{ij} \quad (1.12)$$

Потрібно в області допустимих вирішень системи рівнянь (1.6) і (1.7) знайти рішення, що мінімізує лінійну функцію (1.12).

Таким чином, ми бачимо, що транспортна задача є задачею лінійного програмування. Для її вирішення застосовують також симплекс-метод, але через специфіку задачі тут можна обійтися без симплекс-таблиць. Розв'язок можна отримати шляхом деяких перетворень таблиці перевезень. Ці перетворення відповідають переходу від одного плану перевезень до іншого. Але, як і в загальному випадку, оптимальний розв'язок шукається серед базисних рішень. Отже, ми матимемо справу тільки з базисними (або опорними) планами. Оскільки в даному випадку ранг системи обмежень-

рівнянь рівний $m+n-1$ то серед всіх mn невідомих x_{ij} виділяється $m+n-1$ базисних невідомих, а решта $(m-1) \cdot (n-1)$ невідомих є вільними. У базисному розв'язку вільні невідомі рівні нулю. Зазвичай ці нулі в таблицю не вписують, залишаючи відповідні клітки порожніми. Таким чином, в таблиці перевезень, що представляє опорний план, ми маємо $m+n-1$ заповнених і $(m-1) \cdot (n-1)$ порожніх кліток.

Для контролю треба перевіряти, чи рівна сума чисел в заповнених клітках кожного рядка таблиці перевезень запасу вантажу на відповідній базі, а в кожному стовпці — потребі замовника [цим підтверджується, що даний план є розв'язком системи (1.6)].

Зауваження 1. Не виключаються тут і вироджені випадки, тобто можливість перетворення на нуль однієї або декількох базисних невідомих. Але ці нулі на відміну від нулів вільних невідомих вписуються у відповідну клітку, і ця клітка вважається заповненою.

Зауваження 2. Під величинами c_{ij} , очевидно, не обов'язково мати на увазі тільки тарифи. Можна також вважати їх величинами, пропорційними тарифам, наприклад, відстанями від баз до споживачів. Якщо, наприклад x_{ij} виражені в тоннах, а c_{ij} у кілометрах, то величина S визначається формулою (1.12), є кількістю тонно-кілометрів, що складають об'єм даного плану перевезень. Очевидно, що витрати на перевезення пропорційні кількості тонно-кілометрів і, отже, будуть мінімальними при мінімумі S . В цьому випадку замість матриці тарифів ми маємо *матрицю відстаней*.

Із сказаного в попередньому пункті витікає наступний *критерій оптимальності базисного розв'язку транспортної задачі*: якщо для деякого базисного плану перевезень суми алгебри тарифів по циклах для всіх вільних кліток ненегативні, то цей план оптимальний [1].

Звідси витікає спосіб відшукування оптимального розв'язку транспортної задачі, що полягає в тому, що, маючи деякий базисний розв'язок, обчислюють

суми алгебри тарифів для всіх вільних кліток. Якщо критерій оптимальності виконаний, то даний розв'язок є оптимальним; якщо ж є клітки з негативними сумами алгебри тарифів, то переходять до нового базису, проводячи перерахунок по циклу, відповідному одній з таких кліток. Отриманий таким чином новий базисний розв'язок буде краще початкового – витрати на його реалізацію будуть меншими. Для нового розв'язку також перевіряють здійснимість критерію оптимальності і у разі потреби знову здійснюють перерахунок по циклу для однієї з кліток з негативною сумою алгебри тарифів і так далі.

Через кінцеве число кроків приходять до шуканого оптимального базисного розв'язку.

У випадку якщо суми алгебри тарифів для всіх вільних кліток позитивні, ми маємо єдиний оптимальний розв'язок; якщо ж суми алгебри тарифів для всіх вільних кліток ненегативні, але серед них є суми алгебри тарифів, рівні нулю, то оптимальний розв'язок не єдиний: при перерахунку по циклу для клітки з нульовою сумою алгебри тарифів ми отримаємо оптимальний розв'язок, але відмінний від початкового (витрати по обох планах будуть однаковими).

Розподільним методом є набір наступних дій: спочатку будується початковий опорний план перевезень по одному з вищевикладених правил, а потім послідовно проводиться його поліпшення до отримання оптимального. Для цього для кожної вільної клітки будують замкнутий цикл. Якщо в матриці перевезень міститься опорний план, то для кожної вільної клітки можна утворити і при тому тільки один замкнутий цикл, що містить цю вільну клітку і деяку частину зайнятих кліток.

Тарифи в клітках, що знаходяться в непарних вершинах циклу, беремо із знаком плюс, а в парних — із знаком мінус. По кожному циклу підраховуємо суму алгебри S_{ij} тарифів.

Якщо замкнутий цикл має вигляд: $(i, j) \rightarrow (d_o, j) \rightarrow (d_o, l) \rightarrow (t, l) \rightarrow \dots \rightarrow (u, v) \rightarrow (i, v)$, то $S_{ij} = C_{ij} - C_{kj} + C_{kl} - C_{tl} + \dots + C_{uv} - C_{iv}$, де (i, j) — вільна клітка.

Якщо сума алгебри S_{ij} негативна, то шляхом зміни значень, що стоять в клітках замкнутого циклу, можна отримати план з меншим значенням цільової функції. Критерієм оптимальності при знаходженні мінімуму функції служить позитивність сум алгебри S_{ij} . Якщо вказана вимога не дотримана, план не оптимальний і підлягає поліпшенню.

Обчислення при рішенні транспортної задачі розподільним методом ведуться по наступному алгоритму:

1. початкові дані задачі розташовують в розподільній таблиці;
2. будують початковий опорний план за правилом "північно-західного кута", або за правилом "мінімального елементу", або методом Фогеля; при цьому повинні виявитися зайнятими $r=m+n-1$ кліток. Проте, якщо опорний план є виродженим, то ця умова не дотримується. Для збереження числа зайнятих кліток $r=m+n-1$ незмінним проробляють наступні кроки: у таблиці відшуковують клітку, що має мінімальний тариф і не створюючи циклу із зайнятими клітками, поміщають в неї базисний нуль і вважають її надалі зайнятою. Процес пошуку таких кліток продовжується до тих пір, поки число зайнятих кліток не стане рівним $m+n-1$;
3. проводять оцінку першої вільної клітки шляхом побудови замкнутого циклу і обчислення по цьому циклу величини S_{ij} . Якщо $S_{ij} < 0$, то переходять до наступного пункту алгоритму;
4. переміщають по циклу кількість вантажу, рівну найменшому з чисел, розміщених в парних клітках циклу (у клітках із знаком мінус). Далі повертаються до пункту 3. Якщо $S_{ij} \geq 0$, то оцінюють наступну вільну клітку, і так далі, поки не виявлять клітку з негативною оцінкою. Серед всіх кліток з оцінкою менше нуля потрібно знайти клітку з найбільшим порушенням оптимальності, тобто клітку з найменшою оцінкою. Якщо, нарешті, оцінки всіх вільних кліток виявляються ненегативними, то оптимальне рішення знайдене.

МЕТОДИ РОЗВ'ЯЗАННЯ ЗАДАЧІ ПРО ПРИЗНАЧЕННЯ

Розглянемо деякі методи розв'язування поставленої задачі:

- 1) Метод повного перебору
- 2) Метод перебору по можливим перестановкам
- 3) Угорський алгоритм

Метод повного перебору

Суть методу полягає в повному переборі всіх можливих варіантів розподілу роботи між механізмами і обрання найкращого.

Змінні:

b – масив поточного розподілу, механізму з номером i відповідає робота з номером $b[i]$

$b1$ – масив найкращого знайденого на даний момент розподілу

$z1$ – Найкраще знайдене значення цільової функції

Розглянемо основні процедури в роботі алгоритму.

- 1) `Next_Step` – перехід на наступний розподіл. Спочатку $p=n$. Доки $b[p]=n$ зменшуємо p на 1. Якщо дійшли до початку масиву, тобто $p=0$, то завершаємо роботу алгоритму, виводимо результат, інакше збільшуємо $b[p]$ на 1.

```
procedure next_step(var b:arr);  
var p:integer;  
begin  
  p:=n;  
  while(b[p]=n)and(p>0) do  
  begin  
    b[p]:=1;  
    dec(p);  
  end;  
  if p=0 then End_algo;  
  inc(b[p]);  
end;
```

- 2) `Z(b)` – знаходить значення цільової функції поточного розподілу робіт.

```
function z(var b:arr):integer;
```

```

var k,i:integer;
begin
  k:=0;
  for i:=1 to n do k:=k+c[i,b[i]];
  Result:=k;
end;

```

- 3) Obrobka – процедура обробки поточного розподілу. Складається з 2 етапів. На першому перевіряють можливість поточного розподілу, тобто чи кожна робота відповідає одному механізму. Якщо перший етап пройдений, то переходять до другого. Другий етап – знаходження цільової функції від масиву b. Якщо її значення краще ніж поточне знайдене z1, то заміняємо масив b1 на знайдений, а z1 на Z(b).

```

procedure obrobka;
var i,j:integer;
    t:boolean;
begin
  t:=true;
  for i:=1 to n do
    for j:=1 to n do
      if (i<>j)and(b[i]=b[j]) then t:=false;
    if t and (Z(b)>z1) then
      begin
        for i:=1 to n do b1[i]:=b[i];
        z1:=Z(b1);
      end;
    end;
  end;
end;

```

- 4) End_algo – останній етап, завершення програми, вивід найкращого результату.

Метод перебору виконує свою роботу за n^n кроків, і тому час його роботи на великих n є досить значним.

Метод перебору по можливих перестановках.

В основі методу лежить розгляд всіх можливих перестановок із перших n чисел. Кожній позиції відповідає механізм, а значенню в цій позиції – номер роботи, яку буде виконувати цей механізм.

Розглянемо основну процедуру переходу від однієї ітерації до іншої.

```
procedure next_step(var b:arr);
var p,i:longint;
begin
  p:=n-1;
  while (p>0)and(b[p]>=b[p+1]) do p:=p-1;
  if p=0 then End_algo;

  i:=p+1;
  while (i<n) and (b[i]>=b[p]) do inc(i);

  if (i=n) and (b[i]>=b[p]) then swap(b[p],b[i])
    else swap(b[p],b[i-1]);

  for i:=1 to ((n-p)div 2) do swap(b[p+i],b[n-i+1]);
end;
```

Нехай ми знаходимось на деякій позиції – $(b[1],b[2],\dots,b[n])$. Здійснюється прохід з кінця масиву доки буде виконуватися умова що попередній елемент є більшим за наступний. Якщо дійшли до початку, тобто масив є посортований за спаданням, то закінчуємо метод, бо перебрали всі можливі випадки. Інакше знайшли позицію p таку, що $b[p]<b[p+1]$.

Наступним кроком є відшукування серед всіх елементів, більших за $b[p]$ знаходимо мінімальний $b[k]$. Міняємо значеннями елементи $b[p]$ та $b[k]$, потім впорядковуємо елементи від p до n . В результаті виконання таких кроків отримаємо наступну по порядку лексикографічну перестановку. Проілюструємо метод для випадку $n=5$ та $b=(4\ 2\ 5\ 3\ 1)$.

$(4\ 2\ 5\ 3\ 1) \rightarrow (4\ 2\ 5\ 3\ 1) \rightarrow (4\ 3\ 5\ 2\ 1) \rightarrow (4\ 3\ 1\ 2\ 5)$.

Всіх можливих перестановок буде $n!$.

ВИСНОВКИ

В даній роботі за допомогою угорського алгоритму та перебірної підходу розв'язано задачу про призначення. Мовою програмування для реалізації розглянутих алгоритмів було вибрано Borland Delphi 7. Дана мова програмування є зручною в використанні, витрачає мало ОЗП, та підтримується будь-якою операційною системою сімейства Windows. В роботі наведено інструкцію користувача для програми, яка реалізує угорський алгоритм та перебірні підходи. Показано на контрольних прикладах правильність роботи програми.

Розроблена програма дозволяє провести порівняльну характеристику угорського алгоритму з методом перебору. Параметрами порівняння є затратений час на знаходження розв'язку (в мілісекундах). Результати порівняння виводяться у вигляді графіку. Показано, що зі зростанням розмірності задачі про призначення, ефективність роботи угорського алгоритму зростає експоненціально відносно перебірного підходу.

Список використаної літератури

1. Дональд Кнут Искусство программирования, том 4, выпуск 2. Генерация всех кортежей и перестановок — М.: [Вильямс](#), 2008. — С. 160.
2. А. В. Кузнецов, Н. И. Холод, Л. С. Костевич. [Руководство к решению задач по математическому программированию](#). — Минск: [Высшая школа](#), 1978. — С. 110.
3. Альфред В. Ахо, Джон Э. Хопкрофт, Джеффри Д. Ульман. Структуры данных и алгоритмы. — М.: Вильямс, 2003. — 364 с.
4. Морозов В.П., Шураков В.В. Основы алгоритмизации, алгоритмические языки и системное программирование: Задачник. Учебное пособие. — М.: Финансы и статистика, 1994. — 224 с.
5. Карманов В. Г. Математическое программирование: Учеб. пособие. — 5-е изд., стереотип. — М.: ФИЗМАТЛИТ, 2004. — 264 с.
6. Васильев Ф. П., Иваницкий А. Ю. Линейное программирование. — М.: Изд-во «Факториал», 1998. — 176 с.